20 Fitting with linear algebra techniques

20.1 The method

Apologies for a slight change of notation. What were $\vec{x}$ and $\vec{\mu}(a)$ in Section 19 are now $\vec{y}^m$ and $\vec{y}(\vec{a})$. TO BE FIXED, EVENTUALLY.

We have M measurements: $\vec{y}^m = \begin{bmatrix} y_1^m \\ y_2^m \\ \vdots \\ y_M^m \end{bmatrix}$

with covariance $W = \begin{bmatrix} \sigma_{11}^2 & \sigma_{12}^2 & \cdots & \sigma_{1(M-1)}^2 & \sigma_{1M}^2 \\ \sigma_{21}^2 & \sigma_{22}^2 & \cdots & \sigma_{2(M-1)}^2 & \sigma_{2M}^2 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ \sigma_{(M-1)1}^2 & \sigma_{(M-1)2}^2 & \cdots & \sigma_{(M-1)(M-1)}^2 & \sigma_{(M-1)M}^2 \\ \sigma_{M1}^2 & \sigma_{M2}^2 & \cdots & \sigma_{M(M-1)}^2 & \sigma_{MM}^2 \end{bmatrix}$ (Note, of course $\sigma_{ij}^2 = \sigma_{ji}^2$).

We want to fit a function $\vec{y}(\vec{a}) = \begin{bmatrix} y_1(\vec{a}) \\ y_2(\vec{a}) \\ \vdots \\ y_M(\vec{a}) \end{bmatrix}$ where the N parameters to be determined are $\vec{a} = \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_N \end{bmatrix}$

20.1.1 The fit

The χ^2 can be written in terms of a matrix equation:

$$\chi^2 = [\vec{y}^m - \vec{y}(\vec{a})]^T \;\; W^{-1} \;\; [\vec{y}^m - \vec{y}(\vec{a})] \tag{141}$$

where:

- $[\vec{y}^m - \vec{y}(\vec{a})]^T$ is a $1 \times M$ matrix
- W^{-1} is a $M \times M$ matrix
- $[\vec{y}^m - \vec{y}(\vec{a})]$ is a $M \times 1$ matrix

The matrix W^{-1} is the inverse of the covariance matrix for the measurement uncertainties. In case of no correlations between measurements, W^{-1} is the diagonal matrix:

$$W^{-1} = \begin{bmatrix} 1/\sigma_{11}^2 & 0 & 0 & 0 & 0 \\ 0 & 1/\sigma_{22}^2 & 0 & 0 & 0 \\ 0 & 0 & \ddots & 0 & 0 \\ 0 & 0 & 0 & 1/\sigma_{(N-1)(N-1)}^2 & 0 \\ 0 & 0 & 0 & 0 & 1/\sigma_{NN}^2 \end{bmatrix}$$

The parameters $\vec{a}$ are obtained by minimizing the χ^2. This process is justified by the fact that the χ^2 is (up to a constant) one-half the negative log of the likelihood for observing $\vec{y}^m$ given $\vec{a}$ in the case that the

joint probability distribution function for the y_i^m's is a multivariate Gaussian, as discussed in Section 19. Therefore, minimizing χ^2 is equivalent to maximizing likelihood.

We start by guessing a solution $\vec{a}_0$ and define a new vector $\delta\vec{a} \equiv \vec{a} - \vec{a_0}$.
Next we expand in a Taylor series:

$$y_i(\vec{a}) = y_i(\vec{a_0}) + \sum_{j=1}^{N} \frac{\partial y_i}{\partial a_j}\delta a_j = y_i(\vec{a_0}) + \sum_{j=1}^{N} \frac{\partial y(x_i, \vec{a})}{\partial a_j}\delta a_j$$

This expansion is exact for functions that are linear in $\vec{a}$, i.e., polynomials, but not only, for example $y(x, \vec{a}) = a_1 + a_2x^2 + a_3/x$ is also linear in $\vec{a}$

The expansion can be written compactly as $\vec{y} = \vec{y_0} + A\delta\vec{a}$, where:

- $\vec{y} \equiv \vec{y}(\vec{a})$
- $\vec{y_0} \equiv \vec{y}(\vec{a_0})$
- $A_{ij} \equiv \frac{\partial y_i}{\partial a_j}$ is a $M \times N$ matrix.

Equation 141 for the χ^2 includes a term $\vec{y}^m - \vec{y}(\vec{a})$ which can be rewritten as:

$$\vec{y}^m - \vec{y}(\vec{a}) = \vec{y}^m - \vec{y_0} - A\delta\vec{a} = \delta\vec{y} - A\delta\vec{a}$$

where $\delta\vec{y} \equiv \vec{y}^m - \vec{y_0}$. The χ^2 in equation 141 then becomes:

$$\chi^2 = [\delta\vec{y} - A\delta\vec{a}]^T \quad W^{-1} \quad [\delta\vec{y} - A\delta\vec{a}]$$

$$\chi^2 = \delta\vec{y}^T W^{-1}\delta\vec{y} - \delta\vec{y}^T W^{-1}A\delta\vec{a} - \delta\vec{a}^T A^T W^{-1}\delta\vec{y} + \delta\vec{a}^T A^T W^{-1}A\delta\vec{a},$$

Note that W^{-1} is $M \times M$ and symmetric, so $(W^{-1})^T = W^{-1}$. All the terms are (obviously) 1×1 matrices, *i.e.*, numbers. So I can transpose any term that I like. If I transpose the 3rd term in the sum, since $(W^{-1})^T = W^{-1}$, I get exactly the 2nd term. Thus, I can simplify:

$$\chi^2 = \delta\vec{y}^T W^{-1}\delta\vec{y} - 2\delta\vec{y}^T W^{-1}A\delta\vec{a} + \delta\vec{a}^T A^T W^{-1}A\delta\vec{a} \tag{142}$$

Next I want to minimize χ^2 by setting all $\frac{\partial\chi^2}{\partial a_i} = 0$. At this point it is useful to remind ourselves that

- The only pieces that depend on $\vec{a}$ are the $\delta\vec{a}$ terms.
- $\delta\vec{a} = \vec{a} - \vec{a_0}$, so $\frac{\partial\delta a_i}{\partial a_j} = \delta_{ij}$

Then taking $\frac{\partial\chi^2}{\partial a_i}$ and setting it to zero we get:

$$-2\delta\vec{y}^T W^{-1}A + 2\delta\vec{a}^T A^T W^{-1}A = 0 \tag{143}$$

The 2nd factor of two comes from the fact that there are two $\delta\vec{a}$ factors in the 3rd term of the expression for the χ^2 in equation 142.

Let's keep track of matrix dimensionality (for the sake of sanity). The terms (matrices) in equation 143 have dimensions:

- First term: $(1 \times M) \cdot (M \times M) \cdot (M \times N) = (1 \times N)$

- Second term: $(1 \times N) \cdot (N \times M) \cdot (M \times M) \cdot (M \times N) = (1 \times N)$

Thus equation 143 is a matrix representation of a set of N linear equations for the N parameters $\delta\vec{a}$. The equation can be solved for $\delta\vec{a}$ as:

$$\delta\vec{a}^T A^T W^{-1} A = \delta\vec{y}^T W^{-1} A$$
$$A^T W^{-1} A \delta\vec{a} = A^T W^{-1} \delta\vec{y}$$

$$\boxed{\delta\vec{a} = (A^T W^{-1} A)^{-1} A^T W^{-1} \delta\vec{y}} \tag{144}$$

and

$$\boxed{\vec{a} = \vec{a_0} + \delta\vec{a}} \tag{145}$$

If the function is linear in $\vec{a}$, this is the final solution. If not, one can iterate: change the initial guess $\vec{a_0} \to \vec{a_0} + \delta\vec{a}$ and repeat as many times as needed until $\delta\vec{a}$ is sufficiently close to zero. Or, equivalently, as the χ^2 improvement from one iteration to the next becomes small enough.

20.1.2 The covariance matrix V of the fit parameters

Here I give two ways to obtain the covariance matrix for the fit parameters.

20.1.2.1 Method 1, propagation of errors: Here we propagate the errors on the measurements $\vec{y}^m$, encapsulated in the covariance matrix W, into the fitted $\vec{a}$.

Define

$$G \equiv (A^T W^{-1} A)^{-1} \tag{146}$$

Then, rewrite equation 144 as:

$$\begin{aligned} \delta\vec{a} &= G A^T W^{-1} \vec{\delta y} \\ \delta\vec{a} &= C \vec{\delta y} \end{aligned} \tag{147}$$

with

$$C \equiv G A^T W^{-1} = (A^T W^{-1} A)^{-1} A^T W^{-1}$$

Then, since $\delta\vec{a} = \vec{a} - \vec{a}_0$ and $\delta\vec{y} = \vec{y}^m - \vec{y}_0$:

$$\vec{a} = C\vec{y}^m - C\vec{y}_0 + \vec{a}_0 \tag{148}$$

We now simply apply equation 132 to obtain the covariance matrix of the $\vec{a}$, which we call V, in terms of W the covariance matrix of $\vec{y}^m$. The matrix D defined in equation 131 is simply C. Thus we get:

$$V = CWC^T \tag{149}$$

Then $C^T = (GA^T W^{-1})^T$ and, since W and W^{-1} are symmetric, $C^T = W^{-1} A G^T$.

Going back to equation 149, and noting that $G^{-1} = A^T W^{-1} A$:

$$V = CWC^T = GA^T W^{-1} W W^{-1} A G^T = GA^T W^{-1} A G^T = G G^{-1} G^T = G^T$$

Finally, since V is symmetric:

$$\boxed{\boldsymbol{V = G = (A^T W^{-1} A)^{-1}}} \tag{150}$$

20.1.2.2 Method 2, from the likelihood: As we have seen in Section 17, the inverse covariance matrix is (approximately) given by:

$$(V_{ij})^{-1} = -\frac{\partial^2 \log \mathcal{L}}{\partial a_i \partial a_j},$$

where $\mathcal{L}$ is the likelihood, and the second derivative is calculated at the minimum.

For Gaussian uncertainties $\chi^2 = -2 \log \mathcal{L}$ (up to a constant), therefore:

$$(V_{ij})^{-1} = \frac{1}{2} \frac{\partial^2 \chi^2}{\partial a_i \partial a_j}.$$

In equation 142 we had:

$$\chi^2 = \delta \vec{y}^T W^{-1} \delta \vec{y} - 2 \delta \vec{y}^T W^{-1} A \delta \vec{a} + \delta \vec{a}^T A^T W^{-1} A \delta \vec{a} \tag{151}$$

When taking 2nd derivatives with respect to a_i and a_j, only the last (3rd) term survives. Therefore we have:

$$(V_{ij})^{-1} = \frac{1}{2} \frac{\partial \chi^2}{\partial a_i \partial a_j} = A^T W^{-1} A \tag{152}$$

which gives:

$$\boxed{\boldsymbol{V = (A^T W^{-1} A)^{-1}}} \tag{153}$$

which is identical to equation 150

PS: in deriving equation 153 the factor of $\frac{1}{2}$ in equation 152 is cancelled because of the "double sum" over a's in the 3rd term of equation 151 for χ^2.

20.2 Recipe for fitting with linear algebra techniques

We can summarize the technique developed in Section 20.1 with the following recipe:

1. Guess the parameters $\vec{a}_0$
2. Build a column vector $\vec{y}_0 \equiv \vec{y}(\vec{a}_0)$
3. Build a column vector $\delta \vec{y} \equiv \vec{y}^m - \vec{y}_0$
4. Build a matrix $A_{ij} \equiv \partial y_i / \partial a_j$. The derivatives are calculates at $\vec{a} = \vec{a}_0$
5. Solve for $\delta \vec{a} = (A^T W^{-1} A)^{-1} A^T W^{-1} \delta \vec{y}$
6. Then $\vec{a} = \vec{a}_0 + \delta \vec{a}$
7. If $\vec{y}(\vec{a})$ was linear in $\vec{a}$, the $\vec{a}$ from step 6 is the solution. Otherwise, set $\vec{a}_0 = \vec{a}$, go back to step 2, and iterate.
8. Iterate until you think you have converged well enough. It is up to you to decide what "well enough" means. It could be that $\delta \vec{a}$ becomes close enough to zero, or that the improvement on χ^2 from one step to the next is small enough.
9. The covariance matrix is $V = (A^T W^{-1} A)^{-1}$.

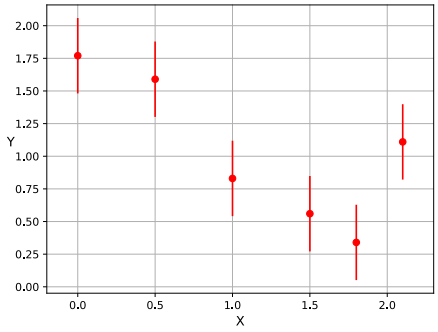


Figure 47: The data to be fit for the example of section 20.3.

20.3 An example

In Fig. 47 I show six data points (x, y) to be fit with the method of Section 20.1. These data points are taken as

$$y(x) = \frac{2}{1+x} + \delta_y$$

and the δ_y's are a set of six random numbers between -0.5 and $+0.5$. I then I fit these data points to a function

$$y(x) = \frac{A}{1+Bx}$$

taking the uncertainties on each data point as $1/\sqrt{12}$. (See Homework 1 to justify this choice). To make life difficult for the fitter, I start the iterations with parameters $A = 12$ and $B = 10$ that are very far away from the true values $A = 2$ and $B = 1$. The code to perform this fit is given in Section 20.3.1.

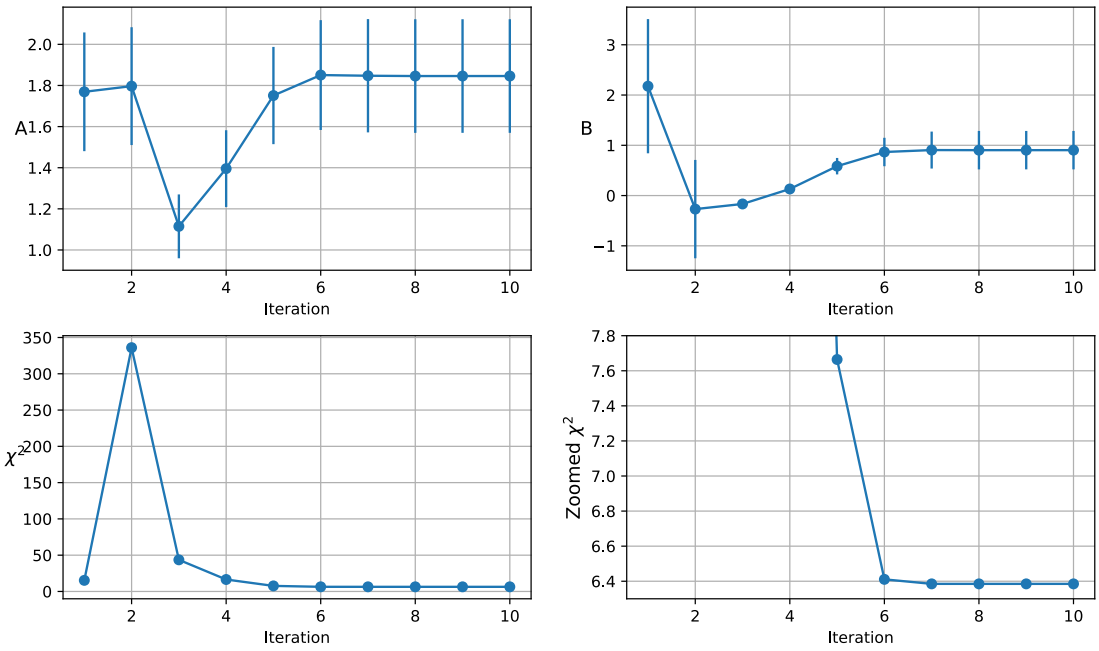


Figure 48: The evolution of the χ^2 and the fitted values of A and B after each iteration.

We see from Fig. 48 that in this low-statistics non-linear problem, with an initial guess quite far from the solution, it takes about six iteration for the fit to converge.

The results of the fit are shown in Table 6 and Fig. 49, where we also show a 1σ uncertainty band on the fitted function (see Section 17.6.1 for instruction on how to construct such bands) as well as the 2D coverage contour (Section 17.5, Tables 3 and 4).

A	1.85 ± 0.28
B	0.90 ± 0.38
ρ	0.71
χ^2	6.39
prob	0.17

Table 6: Results of the fit described in the text. ρ is the Pearson correlation. The quantity *prob* is one minus the CDF of the χ^2 distribution for four degrees of freedom. It can be thought as the p-value for the fit (this quantity is called *prob* in the ROOT toolkit). The number of degrees of freedom is four because there are six data points and two parameters.

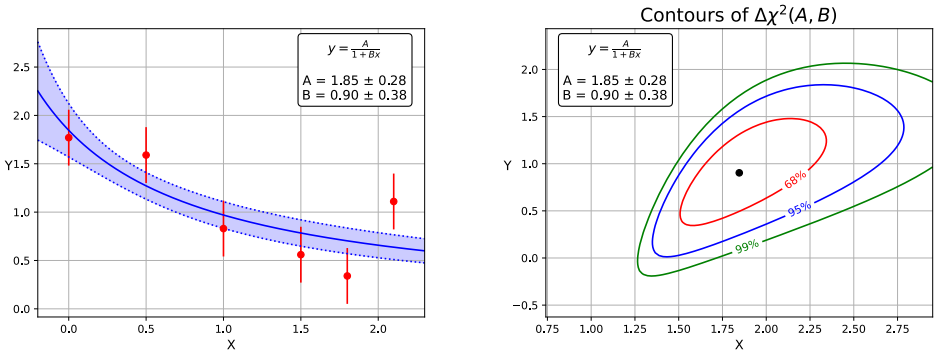


Figure 49: Left plot: fitted function, with 1σ band. Right plot: Contours of $\Delta\chi^2$ in the $(A, B$ plane.

Note that the contours are tilted and not elliptical. The tilt is an indication of the correlation, and being tilted to the right is a sign that the correlation is positive (see Table 6: $\rho = 0.71$). It makes sense for the correlation to be positive: An increase in A would raise the overall level of the curve in the left panel of Fig. 49, and in order to not get too much above the points, B, which is in the denominator as $(1 + Bx)$, would need to increase. The contours are not elliptical because we only have six points and so we are not yet in the asymptotic regime. Still, the shape of the contours are not too far from being elliptical, especially the inner one.

Note that in 1D the χ^2 or the NLL function can always be expanded about its minimum value as $f(a) = f(\hat{a}) + \frac{1}{2}f''(\hat{a})(a-\hat{a})^2 + \frac{1}{6}f'''(\hat{a})(a-\hat{a})^3 + ...$, and similarly in more dimensions. Thus it is natural to find that the departures from the ideal parabolic (elliptical) shapes of the function (contour) grow with the distance from the minimum.

20.3.1 Code

Define the fitting function, the χ^2, and functions to calculate the derivatives

[2]:
```
# The fitting function
# y = p[0] / (1 + p[1]*x)
def f(x,p):
    return p[0]/(1+p[1]*x)


# The derivatives are
```

```
# dy/dp[0] = (1 + p[1]x)
# dy/dp[1] = -p[0]*x / (1+p[1]x)^2
# i = 0 gives df/dp0
# i = 1 gives df/dp1
def dydp(i, p, x):
    temp = 1 + p[1]*x
    if i == 0:
        return 1./temp
    elif i == 1:
        return -p[0]*x/(temp*temp)
    else:
        print("Illegal call to dydp with i = ", i)
    return -1

# the chisq
def getChisq(y0,y,e2):
    return ((y-y0)*(y-y0)/e2).sum()
```

Prepare the data to be fit

[3]:
```
# We make a fake data set y=2+x*x, but
# We smear y a little bit by adding to
# it a random number between -0.5*d and 0.5*d
np.random.seed(918836455)
x = np.array([0.,0.5,1.,1.5,1.8, 2.1])
y = f(x, [2,1])
N = len(x)   # number of points
d = 1.0
# y = y + d*np.random.rand(N) - 0.5*d

# HARDWIRE y VALUES INSTEAD
y  = np.array([1.77, 1.59, 0.83, 0.56, 0.34, 1.11])

# The error in y is d/sqrt(12).
# This is because the variance of a random
# uniform variable bewtween a and b
# is (b-a)**2/12.  Thus d/sqrt(12)...
e2 = np.full(N, d*d)/12.   # variance of y

# The inverse covariance matrix of the y's
W = np.diag(1./e2)

# we will fit as y = p[0] / (1 + p[1]*x)
p  = np.array([12., 10.])  # initial guess (way off!!!!)
y0 = f(x,p)
```

Some empty lists to save the results for 10 iterations.

[4]:
```
# number of iterations
niter = 10

# results saved
Aout   = []
Bout   = []
```

```
chiout = []
eAout  = []
eBout  = []
```

The various matrix multiplications and inversions. The “temp3" array is the covariance matrix. (Bad name, I agree).

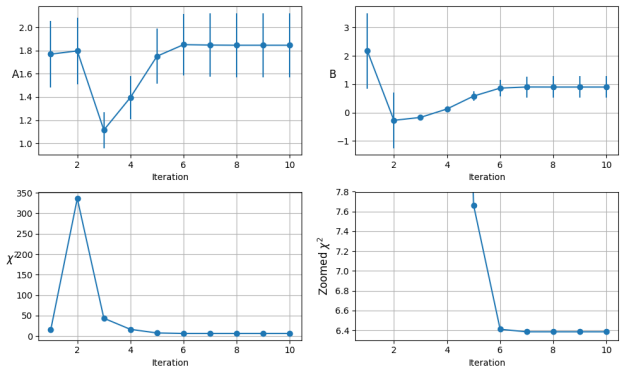
[5]:
```
# matrices A and At are the derivatives of y wrt p
# (the derivative are calculated in a separate function)
for iteration in range(niter):
    At = np.array( [dydp(0,p,x), dydp(1,p,x)] )  # 2xN matrix
    A  = (At.T).copy()                    # Nx2 matrix
    dy = (np.array( [(y-y0),] )).T        # Nx1 column vector
    # find the matrix to be inverted, and invert it
    temp  = np.matmul(At, W)       # 2xN * NxN = 2xN
    temp2 = np.matmul(temp, A)     # 2xN * Nx2 = 2x2
    temp3 = np.linalg.inv(temp2)   # 2x2 ... this is the covariance matrix

    # multiply again
    temp4 = np.matmul(temp3, At)     # 2x2 * 2xN = 2xN
    temp5 = np.matmul(temp4, W)      # 2xN * NxN = 2xN
    dpar  = np.matmul(temp5, dy)     # 2xN * Nx1 = 2x1 column vector

    # the new values of the parameters
    p[0] = p[0] + dpar[0][0]
    p[1] = p[1] + dpar[1][0]

    # the currently fitted value of y
    y0 = f(x,p)

    # the chisq
    chisq = getChisq(y, y0, e2)

    # Save the partial results
    Aout.append(p[0])
    Bout.append(p[1])
    chiout.append(chisq)
    eAout.append( math.sqrt(temp3[0][0]) )
    eBout.append( math.sqrt(temp3[1][1]) )
```

A printout of the final results. The function `chi2.cdf` is the CDF of the χ^2 distribution and comes from the `scipy.stats` package.

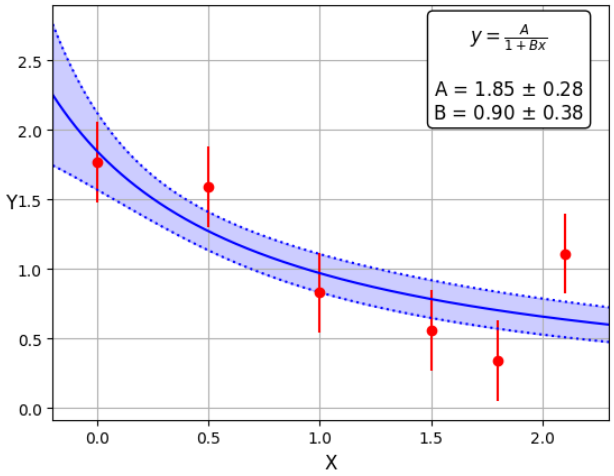
[6]:
```
# Print final results
ndof = len(x)-2
print(" A     = %.2f +/- %.2f" % (Aout[-1], eAout[-1]))
print(" B     = %.2f +/- %.2f" % (Bout[-1], eBout[-1]))
print(" chisq = %.2f" % chiout[-1])
prob = 1-chi2.cdf(chiout[-1], ndof)
print(" prob  = %.2f" % prob)
print(" ndof  =  ", ndof)
# Pearson Correlation
rho = temp3[0][1]/math.sqrt(temp3[0][0]*temp3[1][1])
print(" rho   = %.2f" % rho)
```

```
A     = 1.85 +/- 0.28
B     = 0.90 +/- 0.38
chisq = 6.39
prob  = 0.17
ndof  =   4
rho   = 0.71
```

[7]:
```
# Plot the values of A and B after each iterations
figure = plt.figure(figsize=(10, 6))
iternum = np.linspace(1,10,10, dtype=int)
for i in range(4):
    ax = plt.subplot(2,2,i+1)
    if i==0:
        D = Aout
        E = eAout
        L = "A"
    elif i==1:
        D = Bout
        E = eBout
        L = "B"
    else:
        D = chiout
        E = 10*[0]
        L = r'$\chi^2$'
    ax.errorbar(iternum, D, yerr=E, marker='o')
    rot = 0
    if i==3:
        L = "Zoomed "+ r'$\chi^2$'
        ax.set_ylim(6.3,7.8)
        rot = 90
    ax.grid()
    ax.set_xlabel("Iteration")
    ax.set_ylabel(L, rotation=rot,fontsize='large')
plt.tight_layout()
plt.savefig("Fit_withMatrices_iteration.pdf")
plt.savefig("Fit_withMatrices_iteration.png")
```

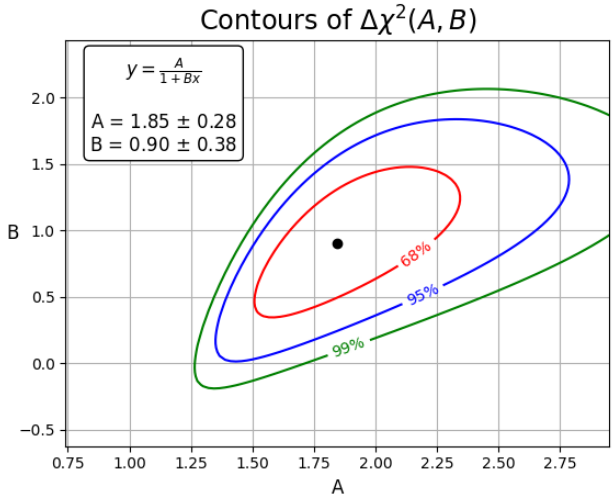


[11]:
```
# Plot the points and the fitted curve
ax=plt.subplot(111)
xmin = x.min()-0.2
xmax = x.max()+0.2
ax.errorbar(x, y, yerr=np.sqrt(e2), linestyle='none', color="red", marker='o')
ax.set_xlim(xmin, xmax)
xpl = np.linspace(xmin, xmax, 1000)
ypl = f(xpl, p)
ax.plot(xpl, ypl, color='blue', linestyle='solid')
ax.grid()
ax.set_xlabel("X", fontsize='large')
ax.set_ylabel("Y", fontsize='large', rotation=0)
ax.grid()

# A text box with the fitted parameters
txtRes = r"$y = \frac{A}{1+Bx}$" + "\n \n"
txtRes = txtRes + "A = " + "%.2f" %Aout[-1]  + " $\pm$ "
txtRes = txtRes + "%.2f" %eAout[-1] + "\n"
txtRes = txtRes + "B = " + "%.2f" %Bout[-1] + " $\pm$ "
txtRes = txtRes + "%.2f" %eBout[-1]
props = dict(boxstyle='round', facecolor='white', alpha=1)
ax.text(0.82, 0.96, txtRes,
       transform=ax.transAxes,
       bbox=props, fontsize='large',
       horizontalalignment='center',
       verticalalignment='top'  )

# and now add a band based on the covariance matrix from the fit
# Note: temp3 is the covariance matrix
dy = np.empty(len(xpl))
for ii in range(len(xpl)):
    xx     = xpl[ii]
    Dt     = np.array( [dydp(0,p,xx), dydp(1,p,xx)] )  # 2x1 matrix
    D      = (Dt.T).copy()          # 1x2 matrix
    blah   = np.matmul(D, temp3)    # 1x2 * 2x2 = 1x2
    dy2    = np.matmul(blah, Dt)    # 1x2 * 2x1 = 1x1
    dy[ii] = math.sqrt(dy2)
ax.plot(xpl, ypl-dy, color='blue', linestyle='dotted')
ax.plot(xpl, ypl+dy, color='blue', linestyle='dotted')
_ = ax.fill_between(xpl, ypl+dy, ypl-dy, color='blue', alpha=.2)

ax.grid()
plt.savefig("Fit_withMatrices_Curve_Result.pdf")
plt.savefig("Fit_withMatrices_Curve_Result.png")
```



[12]:
```
# Now a scan of the chisq
# We calculate the chisq for various values of p[0] and p[1]
# Note: temp3[0][0] and temp3[1][1] are the diagonal
# elements of the covariance matrix, ie, the square of the errors
# on p[0] and [p1],
# We will scan out to 4 sigma
ax2 = plt.subplot(111)
p0min = p[0] - 4*np.sqrt(temp3[0][0])
p0max = p[0] + 4*np.sqrt(temp3[0][0])
p1min = p[1] - 4*np.sqrt(temp3[1][1])
p1max = p[1] + 4*np.sqrt(temp3[1][1])
nscan = 100
p0 = np.linspace(p0min, p0max, nscan)
p1 = np.linspace(p1min, p1max, nscan)
ax2.set_xlim(p0min,p0max)
ax2.set_ylim(p1min,p1max)
z = np.zeros( shape=(nscan,nscan) )    # an emptyt array for now
for i0 in range(0, nscan):
    this_p0 = p0[i0]
    for i1 in range(0, nscan):
        this_p1   = p1[i1]
        yy        = f(x, [this_p0, this_p1])
        z[i0][i1] = getChisq(y,yy,e2) - chisq

# "z" is a map of chisq - chisq_at_minimum
# Correspond to the 68% 95% 99% coverage for 2 parameters
# (2023 PDG Table 40.2)
CS = ax2.contour(p0, p1, z, [2.30, 5.99, 9.21], colors=['red', 'blue', 'green'])
fmt = {}
strs = [ '68%', '95%', '99%' ]
for l,s in zip( CS.levels, strs ):
    fmt[l] = s
ax2.clabel(CS, inline=True, fmt=fmt, fontsize=10)
```

```
# put a point at the best fit
ax2.plot(p[0], p[1], 'ko')

ax2.grid()
ax2.set_xlabel("A", fontsize='large')
ax2.set_ylabel("B", fontsize='large', rotation=0)

ax2.text(1-0.82, 0.96, txtRes,
       transform=ax2.transAxes,
       bbox=props, fontsize='large',
       horizontalalignment='center',
       verticalalignment='top'  )
_ = ax2.set_title(r"Contours of $\Delta\chi^2(A,B)$", fontsize='xx-large')
plt.savefig("Fit_withMatrices_Contours.pdf")
plt.savefig("Fit_withMatrices_Contours.png")
```



20.4 The case of no uncertainties

In this case there is no χ^2 to be defined, and it is usually not what we do in physics, at least not in high energy physics. But it is what is done in many other fields. In this case, instead of minimizing the χ^2, one minimize the sum of squares (SSQ), which we could also write in matrix form using equation 141 for χ^2 and taking W to be the $N \times N$ identity matrix I.

The solution for the fitted parameters, after each iteration, is then given by equations 144 and 145 with $W = I$. The fundamental assumption, rarely if ever stated, is that all measurements are independent, and that the uncertainty on every measurement is the same.

However, in order to give an uncertainty on the fitted parameters one obviously needs an estimate of this common uncertainty to build a W matrix to feed into equation 153. This estimate can come from the fit residual themselves, since the expectation value of SSQ is

$$\mathbf{E}(SSQ) = (M - N)\sigma^2$$

which is then taken to be the observed SSQ in order to extract the value of σ^2 to be inserted into equation 153. (Recall: M is the number of measurements and N is the number of parameters, so $M - N$ is the number of degrees of freedom).

20.5 Bootstrap

An alternative method to estimate the fit uncertainties is "bootstrap with replacement". In this method one builds several data sets by sampling the original M-dimensional dataset. Each new sample consists of M elements picked at random, where a given element of the original dataset could be picked more than once. The new samples are then all processed through the fitting code, and the distribution of resulted fitted parameters are used to estimate the uncertainty. I have never seen this method used in HEX but it is used, for example, in Astrophysics[26].

21 Constrained Fits

There are cases where we want to perform fits with some constraint on the parameters. For example, in the decay $X \to AB$, where A and B are charged particles, we might want to fit the parameters of the A and B tracks simultaneously, with the constraint that they originate from a common point in space. This extra piece of information has several effects:

- It improves the measurement of the A and B momenta.
- It improves the determination of the decay point of the X particle, as compared to, for example, setting it to be the point of closest approach of the two tracks in 3D. This may be useful if, for example, one was interested in measuring the lifetime of X through its decay length.
- Better momentum measurements results in better measurement of the invariant mass of the AB system, and therefore better background rejection against AB pairs that do not arise from the decay of X.

In HEX this procedure is referred to as "vertex-constrained fitting". You will be exploring this in a MC toy example in Homework 3. Sometimes it makes sense to also add the constraint that the invariant mass of AB is equal to the known mass of X. This would then be a "mass and vertex-constrained fit".

While (mass and) vertex-constrained fits are ubiquitous, they are not the only application of constrained fits in HEX The technical formulation of the problem is that given N parameters $a_1, a_2, ..., a_N$ one writes the constraint as $f(\vec{a}) = 0$, where $f(\vec{a})$ is a suitably chosen function, possibly quite complicated, that encapsulates the constraint. There are three ways to tackle this problem.

21.1 Method 1: Elimination of Variables

If $f(\vec{a}) = 0$ is simple enough that can be solved analytically, one can eliminate one of the parameters and reformulate the likelihood or the χ^2 in terms of the remaining $N - 1$ parameters.

For example, suppose the constraint is $f(\vec{a}) = a_2 + 3a_1 - 4a_3 = 0$. Then $a_2 = 4a_3 - 3a_1$ can be eliminated from the fit. At the end of the day, if one was interested in $\hat{a}_2$, it could be computed from $\hat{a}_3$ and $\hat{a}_1$, and its uncertainty could be obtained from propagation of errors.

If there are two or more constraints, and the equations are solvable analytically, one can then eliminate in the same way a second (or third, or...) parameter.

[26] See for example https://ned.ipac.caltech.edu/level5/Wall2/Wal3_5.html

21.2 Method 2: Lagrange Multipliers

To impose constraints, a new parameter, a "lagrange multiplier" λ_i is introduced for each constraint (in the following we take the number of constraints to be P). The χ^2 is modified as $\chi^2 \to \chi^2 + 2\sum \lambda_i f_i(\vec{a})$. The factor of 2 is arbitrary, but (somewhat) convenient, as we will see below.

The χ^2 minimization then occurs with respect to the N parameters $\vec{a}$ and the P parameters $\vec{\lambda}$. The minimization requirement that the partial derivative of the χ^2 with respect to λ_i be equal to zero insures that $\vec{f}_{\,i}(\vec{\hat{a}}) = 0$.

This method is badly suited to numerical minimizations. When looking for minima a numerical method will let $\lambda_i \to -\infty$ rather than converging on $f_i(\vec{\hat{a}}) = 0$. In contrast, the linear algebra method, which is based on conditions such as $\partial\chi^2/\partial\lambda_i$ will naturally lead to $f_i(\vec{\hat{a}}) = 0$. As a consequence, lagrange multiplier methods are best implemented using the approach of Section 20.

To see how this works we go back to equation 143 which applies to the no constraint case:

$$-2\delta\vec{y}^T W^{-1} A + 2\delta\vec{a}^T A^T W^{-1} A = 0$$

This equation was obtained from the conditions $\partial\chi^2/\partial a_j = 0$. I rewrite it by taking the transpose as:

$$-2A^T W^{-1}\delta\vec{y} + 2A^T W^{-1} A\delta\vec{a} = 0$$

which I schematically write as

$$[2A^T W^{-1} A]\ [\delta\vec{a}] = [2A^T W^{-1}\delta\vec{y}] \tag{154}$$

where I use the square-brackets to remind ourselves that these are matrices. The first matrix on the left-hand side has dimensions $N \times N$, the second is $N \times 1$. The matrix on the right-hand-side also has dimensions $N \times 1$.

With the constraints, the terms that arose from $\partial\chi^2/\partial a_j$ acquire additional contributions $2\lambda_r \partial f_r/\partial a_j$. This then leads to the definition of a $P \times N$ matrix with elements

$$D_{rj} = \frac{\partial f_r}{\partial a_j}$$

Further, there are also terms associated with $\partial\chi^2/\partial\lambda_r = 0$. After some algebra, these lead to

$$2\ D_{rj}\delta a_j = -2f_r(\vec{a}_0)$$

The upshot is that the matrix equation 154 becomes, schematically:

$$\begin{bmatrix} [2A^T W^{-1} A] & [2D^T] \\ [2D] & [0] \end{bmatrix} \begin{bmatrix} [\delta\vec{a}] \\ [\vec{\lambda}] \end{bmatrix} = \begin{bmatrix} [2A^T W^{-1}\delta\vec{y}] \\ [-2\vec{f}(\vec{a}_0)] \end{bmatrix} \tag{155}$$

The first matrix on the left-hand side has dimensions $(N+P) \times (N+P)$, the second is $(N+P) \times 1$. The matrix on the right-hand-side is also $(N+P) \times 1$. Equation 155 can be inverted to yield:

$$\begin{aligned} \begin{bmatrix} [\delta\vec{a}] \\ [\vec{\lambda}] \end{bmatrix} &= \begin{bmatrix} [A^T W^{-1} A] & [D^T] \\ [D] & [0] \end{bmatrix}^{-1} \begin{bmatrix} [A^T W^{-1}\delta\vec{y}] \\ [-\vec{f}(\vec{a})] \end{bmatrix} \\ \begin{bmatrix} [\delta\vec{a}] \\ [\vec{\lambda}] \end{bmatrix} &= \begin{bmatrix} [Q] & [D^T] \\ [D] & [0] \end{bmatrix}^{-1} \begin{bmatrix} [A^T W^{-1}\delta\vec{y}] \\ [-\vec{f}(\vec{a}_0)] \end{bmatrix} \end{aligned} \tag{156}$$

Here I defined a new matrix $Q \equiv A^T W^{-1} A$ Comparing with equations 146, and 150, $Q = G^{-1} = V_{\rm unc}^{-1}$, where $V_{\rm unc}^{-1}$ is the covariance matrix for $\vec{a}$ without the constraint. Equation 156 is the fundamental equation to use in the iteration process.

Not surprisingly this is fairly straight-forward extension of the no-constraint case. In that case equation 156 reduces to $\delta\vec{a} = Q^{-1}A^TW^{-1}\delta\vec{y} = GA^TW^{-1}\delta\vec{y}$ which is the same as equation 147.

We can rewrite the inverse matrix in equation 156 schematically as:

$$\begin{bmatrix} [Q] & [D^T] \\ [D] & [0] \end{bmatrix}^{-1} = \begin{bmatrix} [F] & [B^T] \\ [B] & [H] \end{bmatrix} \tag{157}$$

where F is a $N \times N$ matrix and H is a $P \times P$ matrix.

It turns out that F is the covariance matrix for $\vec{a}$ while H is the covariance matrix for $\vec{\lambda}$. Furthermore, the correlations between $\vec{a}$ and $\vec{\lambda}$ are zero.

Note again the correspondence with the unconstrained case. In that case $F = Q^{-1} = G = V_{\rm unc}$.

We are now going to prove the statement that F is the covariance matrix for $\vec{a}$. From equation 156 we have

$$\delta\vec{a} = FA^TW^{-1}\delta\vec{y} - B^T\vec{f}(\vec{a}_0)$$

We apply propagation of errors on the $\delta\vec{y} = \vec{y}^m - \vec{y}_0$. Using the same arguments that followed equation 147, we obtain the covariance matrix V for $\vec{a}$ as follows:

$$\begin{aligned} V =& FA^TW^{-1}\ W\ (FA^TW^{-1})^T \\ V =& FA^TW^{-1}\ W\ W^{-1}AF^T \\ V =& FA^TW^{-1}AF^T \\ V =& FQF^T \\ V =& FQF \end{aligned} \tag{158}$$

where in the last step I used the fact that F is symmetric. To justify this statement, note that by definition of the inverse,

$$\begin{bmatrix} [Q] & [D^T] \\ [D] & [0] \end{bmatrix} \begin{bmatrix} [F] & [B^T] \\ [B] & [0] \end{bmatrix} = \begin{bmatrix} [1] & [0] \\ [0] & [1] \end{bmatrix} \tag{159}$$

and

$$\begin{bmatrix} [F] & [B^T] \\ [B] & [0] \end{bmatrix} \begin{bmatrix} [Q] & [D^T] \\ [D] & [0] \end{bmatrix} = \begin{bmatrix} [1] & [0] \\ [0] & [1] \end{bmatrix} \tag{160}$$

Multiplying the first (pseudo) line of the first matrix by the first (pseudo) column of the second matrix in equation 159 we get

$$\begin{aligned} QF + D^TB &= 1 \\ F^TQ^T + B^TD &= 1 \\ F^TQ + B^TD &= 1 \end{aligned} \tag{161}$$

since $Q = V_{\rm unc}^{-1}$ is symmetric. Doing the same row $\times$ column multiplication on equation 160:

$$FQ + B^TD = 1 \tag{162}$$

Comparing the last line of equation 161 with equation 162, we conclude that $F = F^T$, *i.e.*, F is symmetric.

Next, we multiply the first line of equation 161 from the left bt F:

$$FQF + FD^TB = F \tag{163}$$

The product of the first line and the the second column of equation 160 gives $FD^T = 0$. Substituting into equation 163, we get $FQF = F$, which, using equation 158, gives the desired result

$$\boxed{V = F} \tag{164}$$

The recipe to carry out this constrained fit is then a simple extension of the one given in Section 20.2:

1. Guess parameters $\vec{a}_0$ and $\vec{\lambda}_0$
2. Build a column vector $\vec{y}_0 \equiv \vec{y}(\vec{a}_0)$
3. Build another column vector $\delta\vec{y} \equiv \vec{y}^m - \vec{y}_0$
4. Build matrices $A_{ij} = \partial y_i/\partial a_j$ and $D_{ij} = \partial f_i/\partial a_j$ The derivatives are calculates at $\vec{a} = \vec{a}_0$
5. Build a column vector $\vec{Y} \equiv A^T W^{-1} \delta\vec{y}$
6. Modify $\vec{Y}$ by adding "at the bottom" a column vector $-\vec{f}(\vec{a}_0)$.
7. Build a matrix
$$U = \begin{bmatrix} [A^T W^{-1} A] & [D^T] \\ [D] & [0] \end{bmatrix}$$
8. Calculate $\vec{X} = U^{-1} Y$
9. Extract the first N elements of $\vec{X}$. This is the vector $\delta\vec{a}$. The other elements of $\vec{X}$ are $\vec{\lambda}$.
10. Then $\vec{a} = \vec{a}_0 + \delta\vec{a}$
11. Iterate, if you like, by setting $\vec{a}_0 = \vec{a}$, $\vec{\lambda}_0 = \vec{\lambda}$ and going back to step 2.
12. At the end the covariance matrix for $\vec{a}$ is the $N \times N$ top left corner of U^{-1}.

21.3 Method 3: Penalty Terms

The penalty term method is an approximate method that is well suited to numerical minimizations.

The method consists of adding terms to the χ^2 or the NLL of the form $f_i^2(\vec{a})/s_i^2$. This does not insure that the constraint $f_i(\vec{a}) = 0$ is exactly satisfied. Roughly speaking, it implies that the constrained $f_i(\vec{a}) = 0$ should be satisfied with an accuracy of order s_i. I use the word "should" as a caveat because the penalty terms compete with whatever else is built into the χ^2 or the NLL, and this "whatever else" may fight strongly against satisfying the $f_i(\vec{a}) = 0$ condition. It goes without saying that one ought to choose the value of s_i judiciously.

There are cases when the penalty term is appropriate from first principles. For example, imagine that one wants to constrain the fitted track in a collider experiment to originate from the beamspot in the transverse plane. The beam spot coordinates are (X_b, Y_b) and the beamspot has widths σ_X and σ_Y. Then if (X_t, Y_t) are the coordinates of closest approach of the track to the z-axis, the χ^2 for the track fit might include penalty terms of the form $(X_t - X_b)^2/\sigma_X^2$ and $(Y_t - Y_b)^2/\sigma_Y^2$.

The functional form of these penalty terms has not been chosen at random. If the beamspot is gaussian, then there is a multivariate Gaussian PDF for a track to come from a particular point in the XY plane:

$$f(X_t, Y_t) = \exp\left(-\frac{(X_t - X_b)^2}{2\sigma_X^2} - \frac{(Y_t - Y_b)^2}{2\sigma_Y^2}\right)$$

This likelihood needs to be multiplied to this PDF. When taking the negative log-likelihood, one get the penalty terms discussed above, within a factor of two. But this factor of two gets reabsorbed in the definition of χ^2, see Section 19. Thus the functional form of the penalty terms is motivated by an underlying Gaussian assumption.